Task 05 — OpenAPI / Swagger Documentation

You need to **document all your APIs** with:

1. Endpoint description
2. Parameters
3. Request body
4. Response
5. Error responses
6. Authentication requirements

Since you are using **Spring Boot**, follow these steps in Eclipse.

Step 1 — Add Swagger dependency

Open:

pom.xml

Inside <dependencies>, add:

```
<dependency>
   <groupId>org.springdoc</groupId>
   <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
   <version>2.8.13</version>
</dependency>
```

Save the file.

Wait for Maven to download the dependency.

Step 2 — Create OpenAPI Configuration

In your project:

src/main/java

└── your package

    └── config

Create:

OpenApiConfig.java

Put this code:

```
package com.example.userservice.config;

import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Info;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class OpenApiConfig {

   @Bean
   public OpenAPI customOpenAPI() {

      return new OpenAPI()
            .info(new Info()
                  .title("User Service API")
                  .version("1.0")
```

```
                .description("API documentation for User Service"));
    }
}
```

Step 3 — Document your User Controller

Open your existing:

UserControllerV2.java

Add these imports:

```
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.Parameter;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
```

At the controller level:

```
@Tag(
    name = "Users",
    description = "User management APIs"
)
@RestController
@RequestMapping("/api/v2/users")
public class UserControllerV2 {
```

Step 4 — Document GET All Users

Add the Swagger annotations above your getUsers() method:

```
@Operation(
   summary = "Get all users",
   description = "Returns a list of all users"
)
@ApiResponses({
   @ApiResponse(
      responseCode = "200",
      description = "Users retrieved successfully"
   ),
   @ApiResponse(
      responseCode = "401",
      description = "Unauthorized"
   ),
   @ApiResponse(
      responseCode = "500",
      description = "Internal server error"
   )
})
@GetMapping
public List<UserResponse> getUsers() {

   return Arrays.asList(
      new UserResponse(1L, "Raju", "raju@gmail.com"),
      new UserResponse(2L, "Kiran", "kiran@gmail.com"),
      new UserResponse(3L, "Suresh", "suresh@gmail.com")
```

```
    );
}
```

Step 5 — Document GET User by ID

Add:

```
@Operation(
    summary = "Get user by ID",
    description = "Returns a user based on the user ID"
)
@ApiResponses({
    @ApiResponse(
        responseCode = "200",
        description = "User found successfully"
    ),
    @ApiResponse(
        responseCode = "404",
        description = "User not found"
    ),
    @ApiResponse(
        responseCode = "401",
        description = "Unauthorized"
    )
})
@GetMapping("/{id}")
public UserResponse getUser(
        @Parameter(
            description = "Unique ID of the user",
            required = true
        )
        @PathVariable Long id) {

    return new UserResponse(
        id,
        "Raju",
```

```
        "raju@gmail.com"
    );
}
```

Step 6 — Document POST Request Body

If your project has a POST API, add this:

```
@Operation(
    summary = "Create a new user",
    description = "Creates a new user"
)
@ApiResponses({
    @ApiResponse(
        responseCode = "201",
        description = "User created successfully"
    ),
    @ApiResponse(
        responseCode = "400",
        description = "Invalid request"
    ),
    @ApiResponse(
        responseCode = "409",
        description = "User already exists"
    )
})
@PostMapping
public UserResponse createUser(
        @RequestBody UserRequest request) {

    return new UserResponse(
        1L,
        request.getName(),
        request.getEmail()
    );
}
```

You need these imports:

```
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.PostMapping;
```

Step 7 — Create UserRequest

If you don't already, have it, create:

`UserRequest.java`

```
package com.example.userservice.controller;

public class UserRequest {

    private String name;
    private String email;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }
}
```

Swagger will automatically display:

Request body

```
{
  "name": "Raju",
  "email": "raju@gmail.com"
}
```

Step 8 — Authentication Requirement

If your API uses authentication, add:

```
import io.swagger.v3.oas.annotations.security.SecurityRequirement;
```

Then,

```
@Operation(
    summary = "Get user by ID",
    description = "Returns a user based on the user ID",
    security = {
        @SecurityRequirement(name = "bearerAuth")
    }
)
```

Task 5 can document the authentication requirement once your security configuration exists.

Step 9 — Run the application

In Eclipse:

Right-click project → Run As → Spring Boot App

Wait until you see something similar to:

Started ... in ... seconds

Then open your browser.

Go to:

http://localhost:8080/swagger-ui/index.html

Step 10 — Check Swagger

You should see:
User Service API
Version 1.0

Users

GET /api/v2/users
GET /api/v2/users/{id}
POST /api/v2/users

Click on an endpoint.

You should be able to see:

Description
Parameters
Request body
Responses
Error responses
Authentication